



Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

ЛАБОРАТОРНА РОБОТА № 9
з дисципліни «Технології розроблення програмного забезпечення»
Тема: «Взаємодія компонентів системи»

Виконала:

Студентка групи ІА-31

Соколова Поліна

Мета: Вивчити види взаємодії додатків (Client-Server, Peer-to-Peer, Service oriented Architecture), та реалізувати в проєктованій системі одну із архітектур.

15. E-mail клієнт (singleton, builder, decorator, template method, interpreter, client-server)

Поштовий клієнт повинен нагадувати функціонал поштових програм Mozilla Thunderbird, The Bat і т.д. Він повинен сприймати і коректно обробляти pop3/smtp/imap протоколи, мати функції автонастройки основних поштових провайдерів для України (gmail, ukr.net, i.ua), розділяти повідомлення на папки/категорії/важливість, зберігати чернетки незавершених повідомлень, прикріплювати і обробляти прикріплені файли.

Хід роботи

Короткі теоретичні відомості:

Клієнт-серверна архітектура

Клієнт-серверна архітектура - модель, у якій система поділяється на клієнтів та сервер. Клієнт відповідає за взаємодію з користувачем, формування запитів та відображення результатів. Сервер виконує всю основну логіку: зберігає дані, обробляє запити та повертає відповіді клієнтам.

Існують тонкі та товсті клієнти.

Тонкий клієнт майже всю роботу перекладає на сервер, а сам показує лише інтерфейс - типовий приклад веб-застосунки. Товстий клієнт, навпаки, виконує значну частину логіки локально, зменшуючи навантаження на сервер і дозволяючи працювати навіть без мережі.

Типова клієнт-серверна система складається з трьох частин: клієнт, спільна проміжна частина (моделі, формати запитів), і сервер з бізнес-логікою та базою даних.

P2P-архітектура

Peer-to-peer архітектура - модель, у якій немає центрального сервера, а кожен вузол одночасно може бути і клієнтом, і сервером. Вузли напряму обмінюються даними та ресурсами.

Основна ідея P2P це повна децентралізація: жодна одиниця не керує всією мережею. Усі учасники мають рівні можливості, що робить мережу стійкою до відмов та добре масштабованою.

Цю архітектуру часто застосовують у файлообмінниках (BitTorrent), блокчейні, децентралізованих фінансах, інтернет-телефонії та розподілених обчисленнях. Основні труднощі — забезпечення синхронності, пошуку ресурсів і безпеки.

Сервіс-орієнтована архітектура

Сервіс-орієнтована архітектура - це підхід, де система складається з незалежних сервісів, кожен з яких виконує окрему бізнес-функцію. Сервіси взаємодіють між собою через стандартизовані мережеві інтерфейси HTTP, SOAP або REST.

SOA спирається на слабке зв'язування, повторне використання сервісів та обмін повідомленнями. Сервіси можуть існувати незалежно, оновлюватися по-окремо та використовуватися різними додатками.

SOA підходить для великих підприємств, де потрібно інтегрувати багато різних підсистем без зміни їх внутрішнього коду. Часто використовується разом із сервісною шиною (ESB).

Мікросервісна архітектура

Мікросервісна архітектура - це еволюція SOA, яка будує систему з великої кількості дуже дрібних автономних сервісів. Кожен мікросервіс має власний контекст, власну базу даних, життєвий цикл і розгортається незалежно від інших.

Мікросервіси взаємодіють між собою за допомогою легких протоколів — HTTP/HTTPS, WebSockets, AMQP, MQTT. Завдяки повній автономності мікросервіси забезпечують масштабованість, гнучкість та простоту оновлення великих систем.

Головні складнощі - координація великої кількості сервісів, складні DevOps-процеси, моніторинг та налаштування комунікації.

Шаблон «Template method»

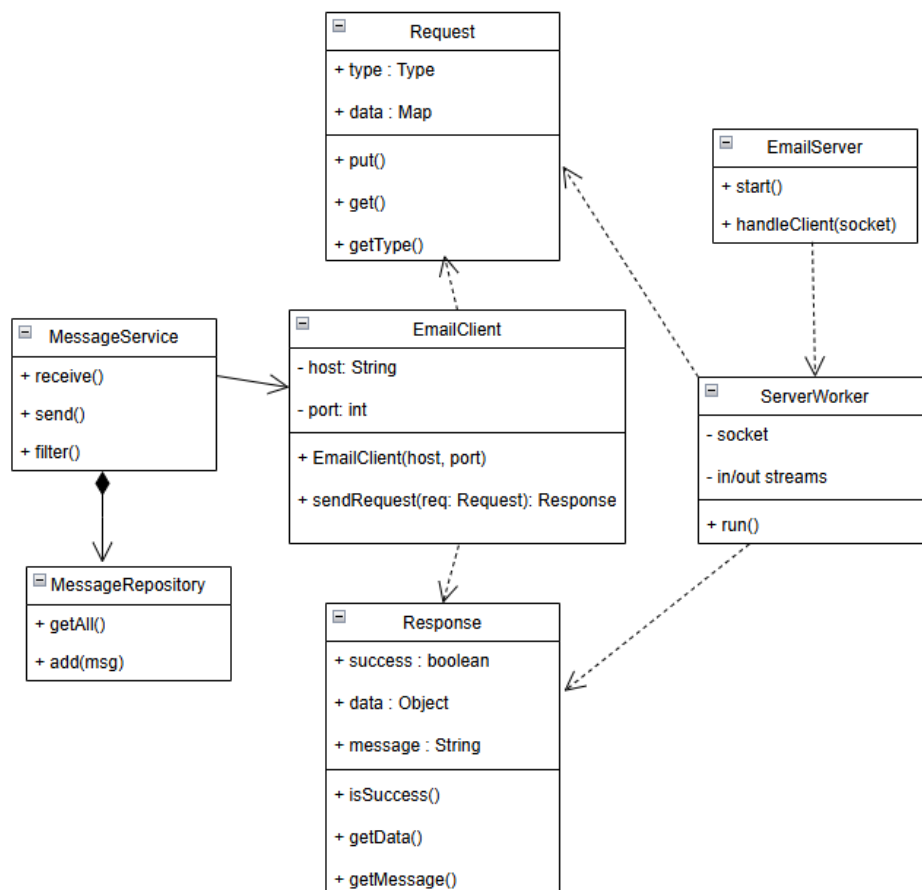


Рис.1 Діаграма класів, яка представляє використання шаблону

На діаграмі класів видно, що **MessageService** керує локальною логікою та викликає **EmailClient** для відправки даних на сервер. **EmailClient** перетворює методи сервісу у мережеві запити **Request/Response**.

EmailServer приймає клієнтів і створює ServerWorker, який обробляє кожне з'єднання окремо. ServerWorker читає Request, виконує дію, взаємодіє з MessageRepository, формує Response і повертає відповідь клієнту.

Таким чином, клієнтська частина відповідає за UI та формування запитів, а серверна - за обробку логіки та доступ до БД. Такий підхід дозволяє централізувати дані, зменшити залежність UI від сховища та підготувати застосунок до масштабування або розгортання в мережі.

```
public class EmailClient { 3 usages

    private final String host; 2 usages
    private final int port; 2 usages

    public EmailClient(String host, int port) { 1 usage
        this.host = host;
        this.port = port;
    }

    public Response sendRequest(Request req) { 3 usages
        try (Socket socket = new Socket(host, port);
            ObjectOutputStream out = new ObjectOutputStream(socket.getOutputStream());
            ObjectInputStream in = new ObjectInputStream(socket.getInputStream())) {

            out.writeObject(req);
            out.flush();

            return (Response) in.readObject();

        } catch (Exception e) {
            e.printStackTrace();
            return new Response( success: false, data: null, e.getMessage());
        }
    }
}
```

1 }

```

public class Request implements Serializable { 14 usages

    public enum Type { 6 usages
        SEND_MESSAGE, 2 usages
        GET_MESSAGES, 2 usages
        FILTER_MESSAGES 2 usages
    }

    private final Type type; 2 usages
    private final Map<String, Object> data = new HashMap<>(); 2 usages

    public Request(Type type) { 3 usages
        this.type = type;
    }

    public Type getType() { 1 usage
        return type;
    }

    public void put(String key, Object value) { 3 usages
        data.put(key, value);
    }

    public Object get(String key) { 3 usages
        return data.get(key);
    }
}
2 }

```

```

public class Response implements Serializable { 11 usages

    private final boolean success; 2 usages
    private final Object data; 2 usages
    private final String message; 2 usages

    public Response(boolean success, Object data, String message) {
        this.success = success;
        this.data = data;
        this.message = message;
    }

    public boolean isSuccess() { no usages
        return success;
    }

    public Object getData() { 2 usages
        return data;
    }

    public String getMessage() {
        return message;
    }
}
3 }

```

```

public class EmailServer {

    private final MessageService messageService = new MessageService(); 1 usage

    public void start(int port) { 1 usage
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Server started on port " + port);

            while (true) {
                Socket client = serverSocket.accept();
                new Thread(new ServerWorker(client, messageService)).start();
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static void main(String[] args) {
        new EmailServer().start( port: 5555);
    }
}
4 }

```

```

public class ServerWorker implements Runnable { 1 usage
    private final Socket socket; 3 usages
    private final MessageService messageService; 4 usages

    public ServerWorker(Socket socket, MessageService service) { 1 usage
        this.socket = socket;
        this.messageService = service;
    }

    @Override
    public void run() {
        try (ObjectInputStream in = new ObjectInputStream(socket.getInputStream());
            ObjectOutputStream out = new ObjectOutputStream(socket.getOutputStream())) {

            Request req = (Request) in.readObject();

            switch (req.getType()) {
                case SEND_MESSAGE -> {
                    Message m = (Message) req.get("msg");
                    messageService.send(m, sign: false, important: false);
                    out.writeObject(new Response( success: true, data: null, message: "Message sent."));
                }
                case GET_MESSAGES -> {
                    int accId = (int) req.get("accountId");
                    List<Message> list = messageService.receive(accId);
                    out.writeObject(new Response( success: true, list, message: "Messages loaded."));
                }
                case FILTER_MESSAGES -> {
                    String q = (String) req.get("query");
                    List<Message> list = messageService.filter(q);
                    out.writeObject(new Response( success: true, list, message: "Filter OK."));
                }
            }
        }
    }
}
5

```

Рис.2 Фрагменти коду по реалізації шаблону

(1– EmailClient, 2 – Request, 3 – Response, 4 – EmailServe, 5 – ServerWorker)

Висновок: у даній лабораторній роботі було реалізовано клієнт-серверну архітектуру. Застосунок отримав окрему мережеву частину, що дозволяє переносити бізнес-логіку та роботу з даними на сервер.

На клієнтському боці працюють контролери інтерфейсу, які приймають дії користувача та формують мережеві запити. Бізнес-логіка клієнта зосереджена у сервісі `MessageService`, що об'єднує роботу з `Decorator`, `Template Method` та `Interpreter` у єдиному доступному API. Загальна (middleware) частина включає універсальні структури `Request` і `Response`, які слугують протоколом обміну між клієнтом і сервером, а також мережевий клієнт `EmailClient`, що відповідає за транспортування цих об'єктів.

Серверна частина побудована на класах `EmailServer` та `ServerWorker`, де кожен клієнт обробляється у власному потоці, а логіка роботи з даними делегована рівню репозиторіїв. На основі отриманого запиту сервер виконує необхідну операцію (надсилання, фільтрацію, отримання повідомлень) і повертає результат у формі `Response`.

Така архітектура чітко розділяє відповідальність між шарами: клієнт займається UI та формуванням запитів, сервер - обробкою та доступом до бази, а middleware - передачею та уніфікацією даних. Завдяки цьому застосунок є масштабованим та легше пристосовується до нових сценаріїв взаємодії в мережевому середовищі.

Відповіді на питання:

1. Що таке клієнт-серверна архітектура?

Це модель, у якій клієнт надсилає запити, а сервер їх обробляє та повертає результат, ролі клієнта й сервера чітко розділені.

2. Розкажіть про сервіс-орієнтовану архітектуру.

SOA - це підхід, де система будується з окремих сервісів, кожен із яких виконує певну функцію та взаємодіє через стандартизовані інтерфейси, найчастіше по HTTP.

3. Якими принципами керується SOA?

Основні принципи: слабке зв'язування, повторне використання сервісів, стандартизовані інтерфейси та обмін через повідомлення.

4. Як між собою взаємодіють сервіси в SOA?

Сервіси обмінюються повідомленнями через HTTP, SOAP або REST API, не маючи прямого доступу до внутрішніх даних один одного.

5. Як розробники взнають про існуючі сервіси і як робити до них запити?

Сервіси реєструються у спеціальному каталозі або документації; доступ до них здійснюється за URL-адресами та оприлюдненими API.

6. У чому полягають переваги та недоліки клієнт-серверної моделі?

Переваги: централізованість, простота оновлень та контрольованість.

Недоліки: залежність від сервера та можливе перевантаження центрального вузла.

7. У чому переваги та недоліки однорангової моделі взаємодії?

Переваги: децентралізація, масштабованість, стійкість до відмов.

Недоліки: складність забезпечення безпеки, синхронізації та пошуку ресурсів.

8. Що таке мікросервісна архітектура?

Це стиль розробки, у якому система складається з невеликих автономних сервісів, що працюють незалежно та мають власний життєвий цикл.

9. Які протоколи використовуються для обміну даними в мікросервісній архітектрі?

Найчастіше HTTP/HTTPS, WebSockets, AMQP, MQTT та інші легкі протоколи.

10. Чи можна назвати підхід SOA, коли між веб-контролерами та шаром доступу до даних створюється шар сервісів?

Ні, це просто шарова архітектура. SOA вимагає автономних розподілених сервісів, які взаємодіють через мережеві інтерфейси, а не внутрішні класи одного застосунку.